

EDOS — Resumen de estudio

Para el examen práctico y la defensa de implementación
Basado en el código entregado en `08-process/`

Tomás Rodeghiero
Sistemas Operativos — UNRC

Resumen

Este documento es la fuente única para preparar el examen práctico y la defensa oral de implementación sobre EDOS, el SO educativo basado en RISC-V utilizado por la cátedra de Sistemas Operativos de la UNRC. La fuente de verdad es el código entregado en `trabajos-entregables/08-process/`; todas las citas a archivo/línea se refieren a ese árbol.

El documento se organiza en cinco partes: (I) visión global de EDOS, arquitectura RISC-V y plataforma; (II) recorrido subsistema por subsistema del kernel con *walkthrough* del código real; (III) tabla comparativa contra xv6 con la justificación pedagógica de cada diferencia; (IV) banco de preguntas razonables que un docente puede hacer en la defensa, con respuesta completa; (V) *cheat sheet* compacto de registros, formato de PTE, secuencia de boot, primer `fork` y comandos útiles de QEMU + GDB.

Importante

Aclaraciones técnicas sobre EDOS. La consigna mencionaba “RISC-V 64 (rv64)” y “paginación Sv39”. El código entregado **no es así**; este documento describe lo que hay en el código:

- EDOS es **RISC-V 32 bits** (`-march=rv32ima -mabi=ilp32, qemu-system-riscv32`). El ensamblador usa `lw/sw`; los registros guardados ocupan 4 bytes cada uno.
- La paginación es **Sv32** (dos niveles: PD 10 b + PT 10 b + offset 12 b = 32). PTE de 32 bits con PPN de 22 b y 10 b de flags (`arch.h:160-213`).
- EDOS **no usa una página *trampoline* separada** como xv6. Los handlers `s_trap` y `u_trap` están en el segmento `.text` del kernel, mapeado en cada `pgtbl` de proceso porque `exec` copia las entradas del `kernel_pgtbl` en el inicio del nuevo page directory (`task.c:386`).
- EDOS sólo define las flags **V, R, W, X, U**. No expone bits A, D ni G en el PTE (`arch.h:175-179`).

Estas diferencias son centrales para la defensa: si te preguntan “Sv39 vs Sv32”, tenés que poder explicar por qué EDOS adoptó Sv32.

Índice

I	Visión global de EDOS	4
1.	Qué es EDOS y para qué sirve	4
2.	Arquitectura objetivo: RISC-V 32	4
2.1.	Modos de privilegio	4
2.2.	Registros CSR relevantes para el SO	5
2.3.	Plataforma: QEMU virt	5
3.	Mapa de memoria física	6
4.	Flujo de arranque end-to-end	6
4.1.	Del firmware/OpenSBI al kernel	6

4.2. Lo que hace boot antes de cualquier C	7
4.3. De supervisor: a kernel_main	7
4.4. Lo que hace kernel_main (kmain.c:10-32)	7
 II Subsistemas del kernel	 8
5. Bootstrap y SMP (arch.s, arch.c, kmain.c)	8
6. Consola y kprintf (console.c, klib.c)	9
7. Spinlocks (spinlock.c)	10
8. Traps e interrupciones (trap.s, trap.c)	11
9. Memoria física: kalloc (kalloc.c)	12
10. Memoria virtual: paginación Sv32 (vm.c, vm.h, arch.h)	13
11. Tareas y scheduler (task.c, task.h)	17
12. Procesos de usuario (task.c + vm.c + arch.s)	20
13. Syscalls (syscall.c, syscall.h)	23
14. Sistema de archivos EFS (efs.c, efs.h, efsfiles.c)	25
15. Programas de usuario (user/)	26
 III Diferencias y similitudes con xv6	 26
16. Tabla comparativa	26
17. Discusión: el porqué pedagógico de cada decisión	27
 IV Banco de preguntas de defensa	 28
18. Conceptuales generales	28
19. Caminá el código de . . .	30
20. ¿Qué pasa si . . . ?	33
21. Cálculos sobre paginación Sv32	35
22. Trade-offs	36
23. Comparativas con xv6	37
24. Detalles finos del código	39
 V Cheat sheet final	 41
25. Registros RISC-V relevantes (1 cara)	41

26.Formato de PTE Sv32	41
27.Secuencia mínima de boot	42
28.Secuencia del primer scheduling de init	42
29.Secuencia de una syscall	42
30.Comandos útiles para QEMU + GDB	42
31.Tabla mental rápida	43

Parte I

Visión global de EDOS

1. Qué es EDOS y para qué sirve

EDOS (*Educational Operating System*) es un kernel mínimo escrito en C y RISC-V assembler, mantenido por la cátedra de Sistemas Operativos de la UNRC y heredado en espíritu de xv6 (MIT). Su objetivo no es competir con un SO real: es **transparentar los mecanismos del SO**. Cada subsistema (gestor de memoria, scheduler, trap handler, sistema de archivos, syscalls) está implementado en la cantidad mínima de líneas que demuestren la idea con claridad, y vive en archivos pequeños y aislados que el estudiante puede leer, modificar y depurar con `gdb` en una tarde.

Objetivos pedagógicos

- **Hacer visible** cada paso desde que se enciende la CPU hasta que un proceso de usuario corre código.
- **Forzar al alumno a tocar** el código del kernel: cada taller agrega un subsistema y rompe algo del anterior para que sea evidente qué hay que arreglar.
- **Trabajar con RISC-V**, una arquitectura abierta, con manuales accesibles y un conjunto de instrucciones lo suficientemente simple para que el ensamblador no oscurezca las ideas.
- **Apoyarse en xv6**, que comparte mucha de la filosofía pero ya en RISC-V; EDOS se permite simplificar a propósito donde xv6 ya es “demasiado” para una cursada.

Diferencias clave con un SO real

- No hay un sistema de archivos persistente: **EFS** es *embebido en el kernel* a través de un `.c` que genera `xxd(efsfiles.c)`.
- No hay `fork/exec` desde el espacio de usuario: el único `fork` conceptual es el que el kernel hace al crear `init`.
- No hay `wait/exit` de uso general: lo cubre el scheduler matando ZOMBIES.
- No hay manejo elaborado de señales, redes, ni soporte de *copy-on-write*.
- La planificación es **round-robin puro**, sin prioridades.

2. Arquitectura objetivo: RISC-V 32

2.1. Modos de privilegio

RISC-V define tres modos:

Modo	Sigla	Rol
Machine	M	Modo más privilegiado. Acceso total al hardware. En EDOS se usa solo durante el <i>boot</i> y para reprogramar el timer del CLINT.
Supervisor	S	Donde corre el kernel de EDOS después del boot. Maneja paginación (vía <code>satp</code>) y traps.
User	U	Donde corren los procesos. Acceso restringido por la MMU y las flags U de las PTEs.

EDOS *delega* casi todas las interrupciones y excepciones de M-mode hacia S-mode con `medeleg` y `mideleg` (`arch.s:41-43`). Sólo la interrupción del timer queda en M-mode (con `mtvec=m_trap`), y

allí se reprograma el `mtimecmp` y se “rebota” como interrupción de software en S-mode levantando `sip.SSIP`.

2.2. Registros CSR relevantes para el SO

CSR	Modo	Rol en EDOS
<code>mhartid</code>	M	ID del hart (CPU lógica). Se lee al inicio y se guarda en <code>tp</code> (<code>arch.s:14</code>).
<code>mstatus</code>	M	Estado global. <code>MPP=01</code> para volver a S-mode con <code>mret</code> (<code>arch.s:33-38</code>).
<code>mtvec</code>	M	Dirección del handler de M-mode (apunta a <code>m_trap</code> , <code>arch.s:58-59</code>).
<code>mepc</code>	M	PC que va a usarse en el próximo <code>mret</code> . Se setea a <code>supervisor</code> (<code>arch.s:69-70</code>).
<code>mie</code>	M	Habilitación de IRQs de M-mode (<code>MTIE=bit 7</code> para timer).
<code>mip</code>	M	Pendientes de M-mode.
<code>mtimecmp</code>	M	Comparador del timer (MMIO en <code>CLINT</code>). Cuando <code>mtime ≥ mtimecmp</code> dispara la IRQ.
<code>medeleg</code>	M	Delega excepciones a S-mode (EDOS pone todo: <code>0xFFFF</code>).
<code>mideleg</code>	M	Delega interrupciones a S-mode (<code>0xFFFF</code>).
<code>pmpcfg0</code>	M	Configuración del PMP para que S/U vean toda la RAM (<code>arch.s:26-30</code>).
<code>sstatus</code>	S	Estado de S-mode. Bit <code>SIE</code> (interrupciones), <code>SPP</code> (modo previo), <code>SPIE</code> .
<code>stvec</code>	S	Dirección del handler de S-mode. EDOS lo cambia entre <code>s_trap</code> (mientras está en kernel) y <code>u_trap</code> (antes de volver a user).
<code>sepc</code>	S	PC en el que ocurrió el trap (donde <code>sret</code> retornará).
<code>scause</code>	S	Causa del trap (interrupción vs. excepción y código).
<code>stval</code>	S	Dato auxiliar (ej.: dirección que generó un page fault).
<code>sscratch</code>	S	“Bolsillo” del kernel. En EDOS guarda la base del kernel-stack del proceso, para que <code>u_trap</code> pueda saltar a él en una instrucción.
<code>satp</code>	S	Page-table address pointer. Bits altos: modo (0 = bare, 1 = Sv32 en rv32). Bits bajos: PPN del directorio.
<code>sie</code>	S	Habilitación de IRQs de S-mode.
<code>sip</code>	S	Pendientes de S-mode. EDOS usa <code>SSIP</code> (software int.) como vehículo para el timer rebotado desde M-mode.
Registros de propósito general (ABI rv32)		
<code>ra</code>	—	Return address.
<code>sp</code>	—	Stack pointer.
<code>tp</code>	—	“Thread pointer”. EDOS lo abusa para guardar el <code>hartid</code> (línea <code>arch.s:14: csrr tp, mhartid</code>).
<code>a0-a7</code>	—	Argumentos y resultado de funciones / syscalls.
<code>s0-s11</code>	—	Callee-saved. Son los únicos guardados por <code>context_switch</code> .
<code>t0-t6</code>	—	Caller-saved. Sólo se salvan en el trap frame.

2.3. Plataforma: QEMU virt

EDOS corre en la máquina virtual `virt` de QEMU, lanzada desde el `Makefile` con:

```
qemu-system-riscv32 -machine virt -bios none -nographic -smp 1 -kernel kernel
```

Mapa de E/S relevante (definido en `arch.h:18-41`):

Dispositivo	Base MMIO	Notas
CLINT	0x02000000	Timer + IPI por hart. <code>mtime</code> en +0xBFF8, <code>mtimecmp[h]</code> en +0x4000+8h.
PLIC	0x0C000000	Controlador de IRQs externas (UART, virtio). Tamaño 4 MiB.
UART	0x10000000	NS16550A. Registros HR (rx/tx), LSR, IER (<code>console.c:6-12</code>).
VIRTIO0	0x10001000	Disco virtio (reservado; EDOS no lo usa todavía).
RAM (kernel + apps)	0x80000000	128 MiB (<code>kernel.ld:35</code>).

Constantes derivadas:

- `T_INTERVAL` = 5 000 000: ticks del `mtime` por cuanto. Aprox. medio segundo en QEMU (`arch.h:27`).
- `UART_IRQ` = 10: número de IRQ del UART en el PLIC.
- `NCPU` = 4: cantidad máxima de harts soportada (estructuras como `cpus_state[]` reservan ese tamaño, aunque el `Makefile` arranca QEMU con `-smp 1`).

3. Mapa de memoria física

El linker script `kernel.ld` fija el layout en RAM:

.text (kernel code)	0x80000000 = <code>__kernel_start</code>
.rodata	
.data	<code>__text_end</code> (alineado a 4KiB)
.bss	<code>__bss</code> , <code>__bss_end</code>
<code>__stack0</code> : 4 stacks de 4KiB	<code>__kernel_end</code>
Pool de páginas libres administrado por <code>kalloc</code>	<code>__mem_end</code> = <code>base</code> + 128MiB

Datos importantes:

- Las primeras dos páginas de cada hart son su stack inicial. `arch.s:17-21` asigna a la CPU `h` el rango [`__stack0` + `h`·4KiB, `__stack0` + (`h`+1)·4KiB).
- Todo lo que está entre `__kernel_end` y `__mem_end` es el **pool de páginas** que `init_kalloc()` encadena en su free-list (`kalloc.c:43-51`).
- Las regiones MMIO (CLINT, UART, PLIC, VIRTIO0) están *fuera* de este rango pero se mapean 1:1 en el `kernel_pgtbl` para que el kernel pueda leerlas/escribirlas con direcciones normales (`arch.c:97-117`).

4. Flujo de arranque end-to-end

4.1. Del firmware/OpenSBI al kernel

QEMU arranca con `-bios none`, así que **no hay OpenSBI** en EDOS: el kernel mismo se carga en 0x80000000 y la primera instrucción que se ejecuta es `boot` (`arch.s:11-72`). Es un caso atípico: en hardware real o con firmware, OpenSBI corre primero en M-mode y delega a S-mode; en EDOS asumimos esa función.

4.2. Lo que hace boot antes de cualquier C

Recorrido del código

arch.s:11-72

1. `csrr tp, mhartid` — guarda el id del hart en `tp`. Es el único lugar donde se lee `mhartid`.
2. `sp = __stack0 + 4KiB·(hartid+1)` — cada hart toma una página de stack distinta. Sin esto, todos escribirían en el mismo stack y el primer `call` corrompería el de los otros.
3. `csrw satp, x0` — **deshabilita la paginación** para correr en modo bare. Se va a habilitar después en `enable_paging` con `satp = 0x80000000 | (kernel_pgtbl»12)`.
4. `pmpcfg0 = 0x1F, pmpaddr0 = -1` — Physical Memory Protection: permite a S y U-mode acceder *a toda* la memoria física. Sin esto, las trampas se dispararían apenas bajáramos a S-mode.
5. Cambiar `mstatus.MPP` de 11 (M) a 01 (S) — el `mret` de más abajo va a volver a S, no a M.
6. `medeleg = mideleg = 0xFFFF` — todas las excepciones e interrupciones se atienden en S-mode. Es el truco que evita escribir handlers de M-mode para cada caso.
7. `sstatus.SIE = 1` — habilitar IRQs de S-mode.
8. `sie ← SEIE | STIE | SSIE` — habilitar las tres clases de IRQs de S-mode (externa, timer, software).
9. `call next_timer_interrupt` — agendar la primera IRQ del timer (programa `mtimecmp`).
10. `mtvec = m_trap` — handler de M-mode (sólo lo usa el timer).
11. Habilitar `mstatus.MIE` y `mie.MTIE`.
12. `mepc = supervisor; mret` — salta a la etiqueta **supervisor** *ya en S-mode*.

4.3. De supervisor: a kernel_main

La etiqueta `supervisor` (`arch.s:77-83`) corre en S-mode con el mismo stack inicial. Lo único que hace es:

```
supervisor:
    la t1, s_trap
    csrw stvec, t1 # de aca en adelante, traps S-mode -> s_trap
    call kernel_main # nunca retorna; arranca el scheduler
```

4.4. Lo que hace kernel_main (kmain.c:10-32)

Recorrido del código

kmain.c:10-32

1. Sólo el hart 0:
 - a) `init_kalloc()` — encadena la free-list de páginas físicas (`kalloc.c:43-51`).
 - b) `init_vm()` — crea el `kernel_pgtbl` y mapea kernel + MMIO 1:1 (`vm.c:6-11`, `arch.c:97-117`).
 - c) `create_process("init")` — el primer proceso de usuario nace acá. Lo veremos en detalle.
 - d) `ready = 1; __sync_synchronize()` — publica el dato a los otros harts.
2. Todos los harts: `while(!ready)` ; y luego `enable_paging()` (activa Sv32 en este hart) y `scheduler()` (no retorna).

Idea intuitiva

La sincronización entre harts es deliberadamente cruda: una variable `volatile int ready` y un `__sync_synchronize()`. Es válido porque sólo el hart 0 escribe y todos los demás giran hasta verla en 1. En xv6 hay un mecanismo equivalente (la espera en `started`).

Parte II

Subsistemas del kernel

5. Bootstrap y SMP (`arch.s`, `arch.c`, `kmain.c`)

Qué resuelve

Llevar a la CPU desde el reset (en M-mode, sin paginación, con `tp` indefinido) hasta un estado en el cual:

- Estamos en S-mode con interrupciones habilitadas.
- Cada hart tiene su propio stack.
- La paginación está montada con el `kernel_pgtbl` (en cada hart).
- El `scheduler` de cada hart elegirá una tarea `RUNNABLE` y le dará CPU.

Estructuras y constantes clave

- `NCPU = 4` (`arch.h:13`).
- `volatile int ready` en `kmain.c:8` — bandera de sincronización inter-hart.

Funciones principales

- `boot` en `arch.s:11-72` — handler de firmware. Configura PMP, MPP, deleg, IRQs y salta a `supervisor:`.
- `supervisor:` en `arch.s:77-83` — instala `s_trap` y llama `kernel_main`.
- `kernel_main(void)` en `kmain.c:10-32` — inicialización (sólo hart 0) y entrega al `scheduler`.
- `init_kalloc()`, `init_vm()`, `create_process("init")`, `enable_paging()`, `scheduler()` (cada uno en su archivo).

Invariantes

- Después de `boot`: cada hart está en S-mode, con su propio stack y con `stvec=s_trap`.
- Sólo el hart 0 ejecuta `init_kalloc/init_vm`. Los demás esperan.
- Cuando un hart pasa el `while(!ready)`, el `kernel_pgtbl` ya existe y es seguro encender `satp`.

Errores frecuentes al modificarlo

- Si `ready` se asigna sin `__sync_synchronize()`, las escrituras al `kernel_pgtbl` pueden no ser visibles a otros harts cuando salen del `busy-wait`.
- Si todos los harts llaman a `init_vm`, se reescribe múltiples veces el page table raíz y se generan dobles liberaciones de páginas.
- Si se omite el ajuste de `sp` por `hartid`, todos comparten stack.

6. Consola y `kprintf` (`console.c`, `klib.c`)

Qué resuelve

Permitir al kernel imprimir mensajes y leer caracteres por la UART, con un `printf` reducido (sólo `%s`, `%d`, `%x`, `% %`) protegido por un spinlock para no entrelazar la salida en SMP.

Funciones clave

- `console_putc(char ch)` en `console.c:24-30` — espera con polling a que `UART_LSR.TX_IDLE` esté seteado y escribe el carácter en `UART_HR`.
- `console_puts(const char *s)` en `console.c:33-39` — toma `console_lock`, recorre la cadena llamando a `console_putc`, libera.
- `console_read_char()` en `console.c:44-52` — chequea `LSR.RX_READY`; si hay byte, lo lee. Si no, devuelve 0. (*No bloquea*; es un polling oportunista.)
- `printf(fmt, ...)` en `klib.c:52-112` — adquiere el lock una sola vez para toda la línea, formatea y llama `putchar` (alias de `console_putc`).

Walkthrough: cómo se imprime un `%x`

Recorrido del código

`klib.c:94-99`

```
case 'x': {
    int value = va_arg(vargs, int);
    for (int i = 7; i >= 0; i--) {
        int nibble = (value >> (i * 4)) & 0xf;
        putchar("0123456789abcdef"[nibble]);
    }
}
```

Es la implementación más simple posible: imprime los 8 nibbles del entero en orden, sin omitir ceros a la izquierda. Es por eso que todos los logs de EDOS tienen el formato `0x80004000`.

Polling vs. interrupciones

EDOS usa **polling** para el TX (esperamos en un `while` a que el bit `TX_IDLE` esté en 1). Para el RX, hay un *hook* de IRQ del UART (`UART_IRQ=10` en el PLIC) pero en la versión 08-process no se activa el path: `console.c` no llama a `console_interrupt_handler`, y `init_external_interrupts/init_external_i` están comentados en `kmain.c:17,29`.

Idea intuitiva

£**Por qué polling en TX?** Porque el UART de QEMU vacía inmediatamente: la espera nunca dura más de un par de ciclos. En hardware real (a 115 200 bps) esto sería inaceptable; ahí se usaría una cola circular + IRQ TX.

Por qué el lock es *caller-side*

`console_putc` **no** toma el lock; sólo `console_puts` y `printf` lo hacen. Esto permite que el `printf` adquiera el lock una vez por línea y no caer en un patrón “char a char con un lock por carácter” que haría que dos `printf` concurrentes se intercalaran a nivel de byte.

7. Spinlocks (spinlock.c)

Qué resuelve

Exclusión mutua **en multiprocesador** sobre estructuras del kernel (tabla de procesos, free-list de páginas, consola, ticks). Un spinlock funciona porque las CPUs comparten memoria coherente y porque RISC-V provee instrucciones atómicas (extensión A).

Tipo y API

```
typedef unsigned int spinlock; // 0 = libre, 1 = tomado

void acquire(spinlock *lk);
void release(spinlock *lk);
```

El tipo es un simple entero. Hay dos primitivas auxiliares no exportadas:

- `push_irq_off(void)` en `spinlock.c:7-17` — deshabilita interrupciones y mantiene un contador de anidamiento (`cpus_state[cpu].noff`) más el estado previo de `sstatus.SIE`.
- `pop_irq_off(void)` en `spinlock.c:19-25` — decrementa `noff` y, sólo si llega a 0, restaura las IRQs si estaban habilitadas antes del primer `push`.

Walkthrough: acquire

Recorrido del código

`spinlock.c:27-44`

```
void acquire(spinlock *lk) {
    push_irq_off(); // (1)
    while (__sync_lock_test_and_set(lk, 1) != 0) // (2)
        ;
    __sync_synchronize(); // (3)
}
```

(1) Antes de cualquier intento, *deshabilitamos interrupciones en esta CPU*. ¿Por qué? Porque si un trap (particularmente el timer) llegara mientras tenemos el lock, podría dispararse `yield()` y dejaríamos a otro proceso girando para siempre sobre nuestro lock.

(2) `__sync_lock_test_and_set` compila a `amoswap.w.aq a5, a5, (s1)` en RISC-V: una operación atómica que escribe 1 en `*lk` y devuelve el valor previo. Mientras devuelva 1, alguien más lo tenía y giramos.

(3) La barrera de memoria garantiza que las lecturas y escrituras dentro de la sección crítica no se reordenen *antes* del lock.

Walkthrough: release

```
void release(spinlock *lk) {
    __sync_synchronize(); // (1) las escrituras de la SC ya son visibles
    __sync_lock_release(lk); // (2) amoswap.w zero, zero, (s1)
    pop_irq_off(); // (3) reactivar IRQs si corresponde
}
```

- (1) Barrera *antes* de soltar el lock.
- (2) Liberación atómica (escribe 0).
- (3) Cierra el `push_irq_off` pareado.

Invariantes y errores típicos

- **Anidamiento:** `acquire/release` pueden anidarse arbitrariamente; las IRQs solo se restablecen cuando `noff` vuelve a 0.
- **Releer sin tener:** si liberás un lock que no tomaste, `noff` se descuenta de más y eventualmente las IRQs se rehabilitan en una sección crítica equivocada.
- **Olvidarse del `push_irq_off`** (versión sin esta protección) lleva a deadlocks clásicos cuando un trap se dispara durante la sección crítica y termina intentando tomar el mismo lock.
- Comparado con un mutex de userland: aquí **no se duerme**. La CPU gira. Por eso sólo conviene para regiones muy cortas. Para esperas largas, EDOS usa `suspend/wakeup` (sleep-locks de hecho contruidos sobre spinlocks).

8. Traps e interrupciones (`trap.s`, `trap.c`)

Qué resuelve

Recibir cualquier evento que cambia el flujo de ejecución (interrupciones del timer, IRQs de dispositivos, syscalls, excepciones como page fault) y dispatcharlo al manejador correspondiente, dejando la CPU en un estado consistente para retornar.

Estructura: `struct trap_frame`

Definida en `arch.h`:100–132, contiene 31 enteros de 32 b (124 B en total):

```
struct trap_frame {
    size_t ra, gp, t0..t6, a0..a7, s0..s11, sp, pc;
} __attribute__((packed));
```

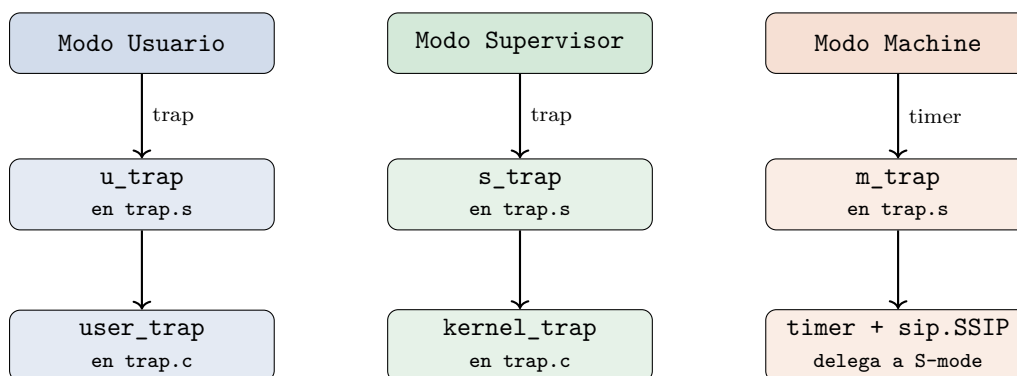
Detalles importantes:

- No incluye `x0` (siempre cero) ni `tp` (lo usamos como hartid permanente).
- El campo `pc` es donde se escribe `sepc` al entrar y desde donde se vuelca antes de `sret`.
- El `sp` guardado es el del modo *previo* (user stack si veníamos de U-mode).
- Los punteros a frame de las syscalls usan helpers como `syscall_number(tf) = tf->a7`.

Los tres handlers de bajo nivel

1. **`s_trap`** (`trap.s`:10–82): traps en S-mode. Reserva 29×4 B en el stack actual y guarda `ra`, `gp`, `t*`, `a*`, `s*` ahí. Llama a `kernel_trap`. Al volver, restaura todo y hace `sret`.
2. **`u_trap`** (`trap.s`:89–140): traps en U-mode. Es más complejo porque hay que cambiar de stack:
 - Intercambia `sp` y `sscratch` (`csrrw sp, sscratch, sp`). Tras eso, `sp` apunta al kernel-stack del proceso y `sscratch` al user-stack.
 - Reserva 31×4 B (un `trap_frame` completo) y guarda los registros, incluido el `sp` de usuario (leído del `sscratch`) y `sepc` en el campo `pc` del frame.
 - Llama a `user_trap`.
3. **`m_trap`** (`trap.s`:200–250): trap de M-mode, sólo para el timer. Reprograma `mtimecmp`, escribe `sip = 2` (setea SSIP, que el S-mode verá como interrupción de software) y hace `mret`.

Diagrama: dónde está cada handler



Walkthrough: el timer en U-mode

Recorrido del código

Camino completo del timer cuando un proceso está corriendo

1. El `mtime` alcanza `mtimecmp`. El hardware levanta una IRQ de M-mode.
2. Al estar deshabilitada la M-mode preemption con `wfi`, la CPU salta a `mtvec = m_trap`.
3. `m_trap` reprograma `mtimecmp` (siguiente cuanto) y levanta `sip.SSIP` (software int. en S-mode). `mret` vuelve al código U-mode interrumpido.
4. Inmediatamente *la propia* CPU ve la IRQ de software en S-mode y atrapa a `stvec = u_trap` (porque estamos en U).
5. `u_trap` swappea `sp/sscratch`, llena el `trap_frame` del proceso, llama `user_trap`.
6. `user_trap` (`trap.c:107-181`) reconoce `cause = TIMER_INTERRUPT` (`0x80000001`, bit de signo prendido $\Rightarrow$ interrupción), llama a `ack_timer_interrupt()` (limpia `sip.SSIP`) y decrementa `task->ticks`.
7. Si llega a 0, llama a `yield()` (queda `RUNNABLE` y vuelve al scheduler).
8. Cuando vuelve la CPU al proceso, `return_to_user_mode()` y `u_trap_ret` restauran el frame.

La función pivote: `return_to_user_mode`

`trap.c:72-101` es la única vía para volver a U-mode. Hace cinco cosas, todas con interrupciones deshabilitadas:

1. Instalar `stvec = u_trap`.
2. Guardar `sscratch = base + 4 KiB` del kernel-stack del proceso. *Esa* dirección la usará `u_trap` para conmutar de pila la próxima vez.
3. Setear `sstatus.SPP=0` y `SPIE=1`, que el `sret` interpretará como “vamos a U-mode con IRQs habilitadas”.
4. Cargar la page-table del proceso con `set_page_table` (que también hace los `sfence.vma`).
5. Saltar a `u_trap_ret(task->trap_frame)` en ensamblador, que mueve `sp = a0`, restaura `sepc` desde el frame, restaura registros y hace `sret`.

9. Memoria física: `kalloc` (`kalloc.c`)

Qué resuelve

Asignar y liberar *páginas físicas* de 4 KiB. Todas las estructuras del kernel que crecen dinámicamente (page tables, kernel stacks, code/data/stack de procesos) las piden a `alloc_page`.

Estructura

Una **free-list** encadenada. Cada página libre lleva en sus primeros bytes un puntero al “próximo” (`struct link`):

```
struct link { struct link *next; };
static struct link *free_list = NULL;
static unsigned int free_pages = 0;
```

Idea intuitiva

Por qué los punteros viven dentro de las páginas libres. Como cada página libre tiene 4 KiB *disponibles*, escribir un puntero en sus primeros 4 bytes no cuesta memoria adicional. El truco se usa en xv6, Linux y en cualquier SO con *slab/freelist* de tamaño fijo.

API

- `void* alloc_page(void)` en `kalloc.c:16-24` — desencola la cabeza de `free_list`, hace `memset(..., 0, PAGE_SIZE)` y la devuelve. Si la lista está vacía, devuelve 0.
- `void free_page(void* pa)` en `kalloc.c:27-40` — verifica alineamiento y rango, encola al inicio de `free_list`.
- `void init_kalloc(void)` en `kalloc.c:43-51` — desde `__mem_end - PAGE_SIZE` bajando hasta `__kernel_end`, llama `free_page` en cada página. Resultado: la free-list queda con la página de menor dirección al frente (LIFO inverso).

Invariantes y errores típicos

- Toda página devuelta por `alloc_page` está **en cero**.
- `free_page` **aborta con panic** si la dirección no está alineada a 4 KiB o cae fuera del rango `[__kernel_end, __mem_end]`.
- **No hay protección de doble free**: si liberás dos veces la misma página, queda dos veces en la free-list y la próxima asignación devuelve un puntero compartido. Ese error no se detecta hasta que se manifiesta en un crash de otra estructura.
- `alloc_page` no toma lock; en SMP debería estar protegido. En el código entregado esta carrera está latente, pero como `Makefile:31` fuerza `-smp 1`, el problema no aparece.

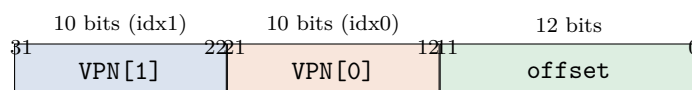
10. Memoria virtual: paginación Sv32 (`vm.c`, `vm.h`, `arch.h`)

Qué resuelve

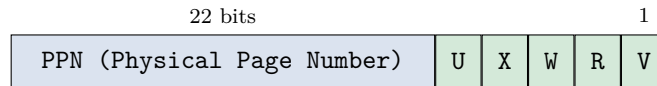
Construir y mantener las tablas de páginas que la MMU usa para traducir direcciones virtuales a físicas. Sin esto, el kernel funcionaría en modo *bare* (acceso directo a RAM física), no podríamos darle un espacio de direcciones independiente a cada proceso ni proteger zonas del kernel del acceso de usuario.

Formato de PTE en Sv32

EDOS usa el régimen Sv32 de RISC-V: tablas de **dos niveles**, página de **4 KiB**. Una dirección virtual de 32 bits se descompone:



Una entrada de tabla (PTE) es de **32 bits** con 22 b de PPN y 10 b de flags:



(EDOS no usa los bits A, D ni G del Sv32 estándar; el RISC-V los ignora si su hardware no los actualiza, lo cual es lo que pasa en QEMU.)

Flags definidas en arch.h:175-179:

- PAGE_V (0x1): válido. Si está en 0, cualquier acceso genera *page fault*.
- PAGE_R (0x2): lectura permitida.
- PAGE_W (0x4): escritura permitida.
- PAGE_X (0x8): ejecución permitida.
- PAGE_U (0x10): accesible desde U-mode (sin esto, el proceso de usuario no puede tocar la página aunque la tenga mapeada).

Macros de traducción (arch.h:188-212)

```
#define page_base_addr(addr) (addr & 0xffff000) // round-down a frontera
#define va_offset(va) (va & 0xfff) // bits 11..0
#define va_index1(va) ((va >> 22) & 0x3ff) // bits 31..22
#define va_index0(va) ((va >> 12) & 0x3ff) // bits 21..12

#define pte_ppn(pte) (pte >> 10) // PPN en bits 31..10
#define pa2ppn(pa) ((pa / PAGE_SIZE) << 10)
#define pte_pa(pte) ((pte >> 10) * PAGE_SIZE)
#define pa2satp(pa) (0x80000000 | (pa >> 12))
```

Atención: el bit alto de `satp` en Sv32 es el bit de modo. EDOS hace `0x80000000 | (pa >> 12)` que en rv32 encodifica “MODE=1, ASID=0, PPN=pa/4K” (Sv32 enabled).

Walkthrough: get_pte y map_page

Recorrido del código

arch.c:48-66 (get_pte, dos niveles)

```
static pte* get_pte(pte* pgtbl, vaddr va, bool create_leaf) {
    uint i = va_index1(va);
    if ((pgtbl[i] & PAGE_V) == 0) { // nivel 1 ausente
        if (create_leaf) { // y queremos crearlo
            paddr pa = (paddr) alloc_page();
            if (!pa) panic("alloc_page() failed!");
            pgtbl[i] = pa2ppn(pa) | PAGE_V; // anclamos hoja
        } else return 0;
    }
    pte* pg0 = (pte*) pte_pa(pgtbl[i]); // base de la hoja
    return pg0 ? &pg0[va_index0(va)] : 0;
}
```

Dos detalles claves:

- En el nivel 1, el PTE **sólo** tiene la flag V prendida (sin R/W/X). Es la convención del Sv32 para indicar “no es una hoja, sigan al próximo nivel”.
- La hoja (pg0) es accedida por dirección física, porque dentro del kernel todo está mapeado 1:1.

arch.c:69-73 (map_page)

```
void map_page(pte* pgtbl, vaddr va, paddr pa, uint flags) {
    pte* entry = get_pte(pgtbl, va, true);
    *entry = pa2ppn(pa) | flags | PAGE_V;
}
```

Una línea para crear la PTE final. Las flags incluyen los permisos R/W/X/U que pasa el caller.

Construcción del kernel_pgtbl

Recorrido del código

vm.c:6-11 (init_vm) y arch.c:97-117 (map_kernel_memory)

```
void init_vm(void) {
    extern pte* kernel_pgtbl;
    kernel_pgtbl = (pte*) alloc_page(); // 1 página = directorio raíz
    map_kernel_memory(kernel_pgtbl);
}

void map_kernel_memory(pte* pgtbl) {
    address ktext_start = (address) __kernel_start;
    uint ktext_size = (uint)(__text_end - __kernel_start);
    address kdata_start = (address) (__text_end);
    uint kdata_size = (uint) (__mem_end - __text_end);

    map_region(pgtbl, UART, UART, PAGE_SIZE, PAGE_R | PAGE_W);
    map_region(pgtbl, VIRTIO0, VIRTIO0, PAGE_SIZE, PAGE_R | PAGE_W);
    map_region(pgtbl, PLIC, PLIC, PLIC_SIZE, PAGE_R | PAGE_W);
    map_region(pgtbl, ktext_start, ktext_start, ktext_size, PAGE_R | PAGE_X);
    map_region(pgtbl, kdata_start, kdata_start, kdata_size, PAGE_R | PAGE_W);
}
```

Cinco regiones **idénticas en virtual y físico** (1:1):

1. UART (0x10000000): R/W.
2. VIRTIO0 (0x10001000): R/W.
3. PLIC (0x0c000000, 4MiB): R/W.
4. Kernel .text: R+X (no W para que un bug del kernel no pueda sobrescribir su propio código).
5. Kernel .data + .bss + stacks + pool de páginas: R+W (no X).

Ninguna de estas tiene el bit U, así que los procesos no pueden tocarlas.

Cambio de espacio de direcciones (arch.h:420-425)

```
static inline void set_page_table(pte* pgtbl) {
    __asm__ __volatile__ ("sfence.vma"); // invalida TLB anterior
    w_satp(pa2satp((size_t) pgtbl)); // cambia raíz
    __asm__ __volatile__ ("sfence.vma"); // invalida TLB nueva
}
```

Las dos `sfence.vma` son críticas. Sin ellas, la MMU podría seguir usando entradas cacheadas que apuntan a la tabla anterior y toda la traducción quedaría mal.

Resumen de funciones de vm.h

- `init_vm()` — crea `kernel_pgtbl`.
- `map_region(pgtbl, va, pa, size, flags)` — mapea un rango (vm.c:14-25).
- `unmap_region(pgtbl, va, size, free)` — desmapea rango (vm.c:28-36).
- `copy_from_user(pgtbl, dst, src_va, n)` y `copy_to_user` — traducen la VA de usuario a su PA kernel-visible vía `va2kernel_address` y hacen `memcpy`.

Errores típicos en VM

- Olvidar `sfence.vma`: traducción impredecible tras cambiar `satp`.

- **No prender PAGE_U** en páginas del proceso: el kernel las ve OK, pero el proceso obtiene page fault al primer acceso.
- **No copiar kernel_pgtbl en exec**: el proceso pierde los mapeos del kernel y, cuando entre a un trap, s_trap/u_trap no estarán en su espacio $\Rightarrow$ panic.

11. Tareas y scheduler (task.c, task.h)

Qué resuelve

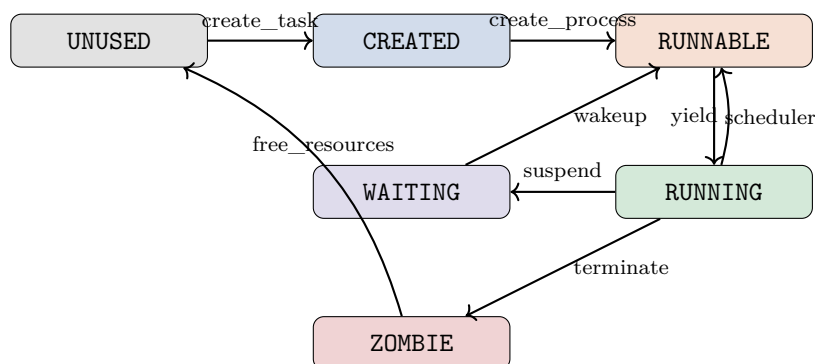
Multiplexar la CPU entre varias *tareas*. Una *tarea* en EDOS es la abstracción común que cubre tanto threads de kernel como procesos de usuario; la diferencia es que un proceso tiene `pid > 0` y un `pgtbl` distinto del `kernel_pgtbl` (task.h:67-68: `is_process`).

Estructura struct task (task.h:23-39)

```
struct task {
    uint tid; // task id (siempre)
    uint pid; // process id (0 si kernel-thread)
    char name[81]; // nombre amistoso
    int state; // UNUSED, CREATED, RUNNABLE, ...
    struct task *parent; // padre (NULL para init)
    bool killed; // marca de muerte
    int exit_code;
    struct context ctx; // ra, sp, s0..s11 (callee-saved)
    int ticks; // cuanto restante (QUANTUM=2 inicial)
    uint8 *kstack; // base del kernel stack (4 KiB)
    uint64 sleep_ticks; // contador para sleep(n)
    int cpu_id; // CPU donde corre, -1 si no corre
    void *wait_condition; // canal de espera (suspend)
    pte *pgtbl; // page table del proceso
    spinlock lock;
};
```

Tabla global: `static struct task tasks[TASK_MAX]` con `TASK_MAX = 16` (task.h:7, task.c:17).

Máquina de estados



(EDOS define también `TERMINATED` en task.h:15, pero en la práctica no la usa el flujo principal: las tareas pasan directo de **RUNNING** a **ZOMBIE**.)

Walkthrough: scheduler (task.c:90-118)

Recorrido del código

```
void scheduler(void) {
    int cpu_id = cpuid();
    cpus_state[cpu_id].task = NULL;
    while (1) {
        enable_interrupts();
        for (int i=0; i<TASK_MAX; i++) {
            acquire(&tasks[i].lock);
            if (tasks[i].state == ZOMBIE) {
                free_resources(&tasks[i]); // libera kstack y marca UNUSED
            } else if (tasks[i].state == RUNNABLE) {
                struct task* next = &tasks[i];
                next->state = RUNNING;
                next->cpu_id = cpu_id;
                next->ticks = QUANTUM;
                cpus_state[cpu_id].task = next;

                context_switch(&cpus_state[cpu_id].ctx, &next->ctx);

                // cuando next cede, volvemos acá
                next->cpu_id = -1;
                cpus_state[cpu_id].task = NULL;
            }
            release(&tasks[i].lock);
        }
    }
}
```

Observaciones:

- Cada CPU tiene su propio “thread de scheduler” con contexto en `cpus_state[cpu].ctx`.
- El recorrido es **round-robin sobre el arreglo `tasks[]`**, no por prioridad ni por cola de listos separada.
- Al hacer `context_switch`, el “ra” guardado del scheduler le permite volver a la línea siguiente cuando la tarea ceda.
- Mientras tiene la CPU una tarea, su lock está tomado (**importante**: `yield` libera tras el `sched`, ver abajo).

yield, sched y la pareja con scheduler

```
/* task.c:140-158 */
void yield(void) {
    struct task *t = cpus_state[cpuid()].task;
    acquire(&t->lock);
    t->state = RUNNABLE;
    sched(t); // salta al scheduler, manteniendo el lock
    release(&t->lock); // al volver, libera el lock
}

/* task.c:126-137 */
static void sched(struct task* current) {
    int cpu_id = cpuid();
    int irq_status = cpus_state[cpu_id].irq_enabled;
    context_switch(&current->ctx, &cpus_state[cpu_id].ctx);
    cpus_state[cpuid()].irq_enabled = irq_status;
}
```

La danza del lock:

1. `yield` entra y *toma* el lock de la tarea.
2. Cambia `state` a `RUNNABLE`.
3. Llama a `sched`, que hace `context_switch` al scheduler.
4. El scheduler “retorna” en el medio de `for`, donde luego va a `release(&tasks[i].lock)`: ese es *nuestro* lock.
5. Más tarde, otro `context_switch` nos trae de vuelta dentro de `sched` → `yield` → `release(&t->lock)`: ahora *el scheduler nos liberó* y nosotros liberamos también. Cuidado: la cuenta de `noff` (`push_irq_off`) debe alinearse.

Walkthrough: `create_task`

Recorrido del código

`task.c:53-79`

```
struct task* create_task(char* name, task_function f) {
    struct task *task = NULL;
    for (int i = 0; i < TASK_MAX && !task; i++) {
        acquire(&tasks[i].lock);
        if (tasks[i].state == UNUSED) {
            task = &tasks[i];
            memset(&task->ctx, 0, sizeof(struct context));
            task->ctx.ra = (address) f; // (a)
            task->kstack = alloc_page(); // (b)
            task->ctx.sp = (address) (task->kstack + PAGE_SIZE);
            task->tid = ++last_tid;
            task->pid = 0;
            task->pgtbl = kernel_pgtbl; // (c)
            task->state = CREATED;
            strcpy(task->name, name);
        }
        release(&tasks[i].lock);
    }
    return task;
}
```

(a) El truco está acá: configuramos el “return address” del contexto guardado para que apunte a `f`. Cuando el scheduler haga `context_switch(&scheduler_ctx, &task->ctx)`, lo último que carga es `ra = f`, e inmediatamente hace `ret` (la instrucción de retorno de `context_switch` en `arch.s:122`). Como `ret` en RISC-V es `jr ra`, terminamos saltando a `f` “como si `f` hubiera sido llamada”.

(b) Reservamos una página para kernel stack y dejamos `sp` en el tope (RISC-V crece para abajo).

(c) Tarea de kernel: comparte el `kernel_pgtbl`. Un proceso va a tener su propio `pgtbl`, asignado más tarde por `exec`.

suspend, wakeup, sleep

Recorrido del código

task.c:163-183

```
void suspend(void* condition, spinlock* lk) {
    struct task* task = current_task();
    acquire(&task->lock);
    release(lk); // liberamos el lock del recurso
    task->wait_condition = condition;
    task->state = WAITING;
    sched(task); // ceder
    task->wait_condition = 0;
    release(&task->lock);
    acquire(lk); // reasumimos el lock del recurso
}
```

Es el patrón clásico (xv6, Linux): *atomically release+sleep*. El truco para que el **wakeup** no se pierda es que **lk** se libera *con task->lock tomado*; el **wakeup**, cuando recorre la tabla, intenta tomar **task->lock**, así que no puede despertarnos antes de que ya estemos en **WAITING** dentro de **sched**.

wakeup (task.c:186-200) recorre todas las tareas, toma su **lock** y cambia a **RUNNABLE** las que coincidan en **wait_condition**.

sleep(n) (task.c:203-212) es un caso particular: deja en **sleep_ticks** el contador y se suspende con condición **&ticks**. El **inc_ticks** en cada timer decrementa **sleep_ticks** y, al llegar a 0, pasa a **RUNNABLE**.

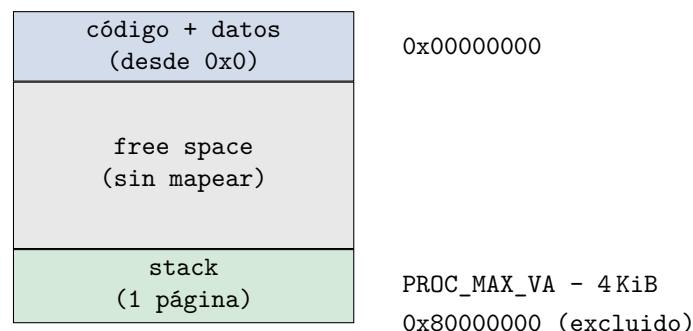
Errores típicos en task.c

- Olvidar el **release(&t->lock)** en algún path: deja el lock tomado para siempre.
- Hacer **wakeup(t)** sin tener **t->lock** (en EDOS lo toma adentro). Si lo hicieras desde fuera con el lock ya tomado, deadlock.
- Cambiar el **state** fuera del lock.
- Asumir que después de **sched(t)** estamos en la misma CPU. Podemos haber migrado.

12. Procesos de usuario (task.c + vm.c + arch.s)

Espacio de direcciones del proceso

PROC_MIN_VA = 0, **PROC_MAX_VA** = 0x80000000 (task.h:19-20). El stack va al final, una sola página:



Creación del primer proceso

Recorrido del código

task.c:424-441 (create_process)

```
bool create_process(char *path) {
    struct task* t = create_task(path, start_process); // (a)
    if (!t) panic("Create process: No tasks free slot!");

    t->pid = ++last_pid; // (b)
    if (!exec(t, path, 0)) { // (c)
        release(&t->lock);
        return false;
    }
    t->state = RUNNABLE; // (d)
    return true;
}
```

- (a) Crear un task con `start_process` como “ra”. Cuando el scheduler lo elija por primera vez, `context_switch` le entregará el control a `start_process`.
- (b) Asignar PID (`++last_pid`).
- (c) `exec` carga el programa.
- (d) `RUNNABLE`: ya es elegible.

Walkthrough: exec

Recorrido del código

task.c:376-421

```
bool exec(struct task* task, char* path, char* args[]) {
    struct trap_frame *tf = task_trap_frame_address(task);

    pte* new_pgtbl = (pte*) alloc_page();
    if (!new_pgtbl) goto error;

    memcpy(new_pgtbl, kernel_pgtbl, PAGE_SIZE); // (1)
    if (!load_program(new_pgtbl, path)) goto error; // (2)

    uint8 *ustack = (uint8 *) alloc_page();
    if (!ustack) goto error;
    map_page(new_pgtbl, PROC_MAX_VA - PAGE_SIZE, (paddr) ustack,
             PAGE_R | PAGE_W | PAGE_U); // (3)

    vaddr sp = setup_main_args(task, ustack + PAGE_SIZE, args); // (4)

    tf->sp = sp; // (5)
    tf->pc = 0; // (6)

    if (is_process(task)) { // (7)
        unmap_region(task->pgtbl, 0, PROC_MAX_VA, true);
        free_page(task->pgtbl);
    }
    task->pgtbl = new_pgtbl;
    task->ctx.sp = (size_t) tf; // (8)
    return sp;
error:
    free_page(new_pgtbl);
    return false;
}
```

(1) `memcpy(new_pgtbl, kernel_pgtbl, PAGE_SIZE)`. Copia el directorio raíz del kernel en el del proceso. Las entradas de hoja del kernel (sus PTs internas) son compartidas con toda otra page table de proceso. Por eso un page fault en código de `s_trap` jamás ocurre: está mapeado en cualquier proceso.

(2) `load_program` (task.c:291-313) hace `efs_file(path)`, asigna páginas para todo el contenido y las mapea con R|W|X|U a partir de VA=0.

(3) Mapea una sola página de stack con permisos R|W|U en `PROC_MAX_VA - 4 KiB`. Sin U el proceso no podría ni escribir su SP.

(4) `setup_main_args` (task.c:335-363) copia los strings de `args[]` al stack y arma el array `argv[]`. Llena `tf->a0 = argc` y `tf->a1 = &argv`.

(5)(6) Punto de retorno a usuario: `pc=0` (entry point del programa) y `sp = top del stack`.

(7) Si el task ya era un proceso (caso de un `exec` sobre un proceso vivo), desmapea su anterior pgtbl y libera.

(8) `task->ctx.sp = (size_t) tf`. Esto es elegante: deja el kernel-mode SP del task apuntando al trap frame. Cuando el scheduler haga `context_switch` a este task y caigamos en `start_process`, `sp` ya estará sobre el trap frame listo para que `return_to_user_mode` lea `task->kstack + PAGE_SIZE - sizeof(tf)`.

start_process y la primera vuelta a U-mode

Recorrido del código

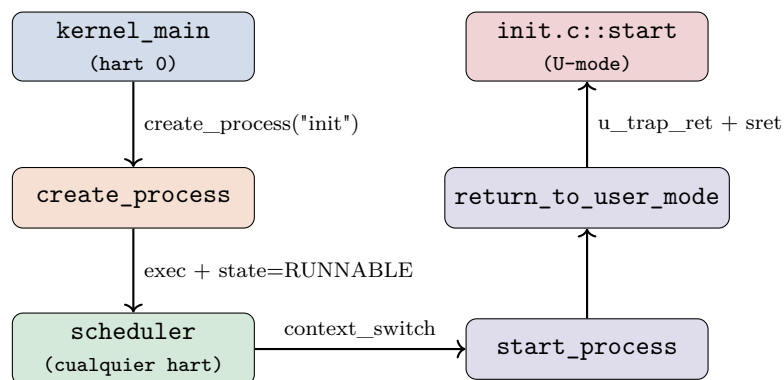
task.c:366-372

```
static void start_process(void) {
    struct task *task = current_task();
    release(&task->lock); // soltamos el lock que tomó el scheduler
    return_to_user_mode(); // configurar CSRs y u_trap_ret
}
```

Recordá: el *scheduler* cuando hace *context_switch* a un task RUNNABLE ya tiene *task->lock* tomado. La “promesa” es que la tarea, al recibir el control, va a soltarlo.

Después de *return_to_user_mode*, el flujo termina en *u_trap_ret* (trap.s:150-192) que pone *sp* al trap frame, restaura los registros y hace *sret*. El *sret* salta a *tf->pc = 0*, que es la entry *start* del binario de usuario (definida primero en *user.ld* con *ENTRY(start)*).

Diagrama: ciclo de vida de un proceso



13. Syscalls (syscall.c, syscall.h)

Convención

Los binarios de usuario implementan cada syscall en *user/usys.s* cargando el número en *a7* y haciendo *ecall*:

```
.global getpid
getpid:
    li a7, 1 # SYS_GETPID
    ecall
    ret
```

Los argumentos viajan en *a0-a6*. El resultado vuelve en *a0*.

Tabla de syscalls

#	Nombre	Comportamiento
0	exit(int code)	Marca killed=true; el trap handler lo cosechará en terminate.
1	getpid()	Devuelve task->pid.
2	console_puts(s)	Traduce la VA del string a kernel y hace printf("%s", ...).
3	console_putc(c)	console_putc(arg0).
4	console_getc()	Lee 1 carácter (no bloqueante; devuelve 0 si no hay).
5	sleep(n)	Suspende la tarea por n ticks.

Walkthrough: el dispatcher (syscall.c:73-84)

Recorrido del código

```
void syscall(struct task *task) {
    struct trap_frame *tf = task_trap_frame_address(task);
    uint n = syscall_number(tf); // tf->a7
    if (n < SYSCALLS && syscalls_table[n]) {
        int result = syscalls_table[n](task);
        syscall_put_result(tf, result); // tf->a0 = result
    } else {
        syscall_put_result(tf, -1);
    }
}
```

Notar que el `syscall` escribe directamente en el trap frame. Cuando `user_trap` retorna y `u_trap_ret` restaura registros, `a0` contiene el resultado.

Después de retornar, `user_trap` llama a `skip_trap_instruction(tf)` (`arch.h:333`) que hace `tf->pc += 4`. Sin esto, el `sret` volvería al mismo `ecall` y caeríamos en un loop infinito.

Validación de punteros

Importante: `sys_console_puts` (`syscall.c:22-29`) traduce la VA del usuario a una dirección kernel:

```
int sys_console_puts(struct task *task) {
    struct trap_frame *tf = task_trap_frame_address(task);
    size_t str = syscall_arg(tf, 0);
    char *kaddr = (char *) va2kernel_address(task->pgtbl, (vaddr)str);
    printf("%s", kaddr);
    return 0;
}
```

Importante

La validación en EDOS es **mínima**: si `kaddr` resulta 0 (página no mapeada en el proceso), el `printf("%s", 0)` va a crashear. Tampoco se chequea longitud, alineamiento ni que toda la cadena viva en una región accesible. Esto es deliberado: EDOS muestra el *esqueleto*, no la validación industrial. En xv6 lo hace `argstr/argint/fetchaddr`.

14. Sistema de archivos EFS (efs.c, efs.h, efsfiles.c)

Qué provee

Un sistema de archivos **en RAM**, embebido en el binario del kernel, de sólo lectura. Cada archivo es un arreglo de bytes con un nombre y un tipo (EFS_FILE_PROGRAM o EFS_FILE_DATA).

Tabla de archivos

```
struct file {
    char *name;
    unsigned char type; // PROGRAM o DATA
    unsigned char *data;
    unsigned int length;
};
extern struct file efs_files_table[];
```

efsfiles.c (generado por user/mkefs.sh con xxd -i) contiene:

- unsigned char init_bin[] — bytes binarios del ejecutable init.
- unsigned char README[] — contenido del README.
- efs_files_table[] = {(init, PROGRAM, init_bin, len), (README, DATA, README, len), {0}}.

API

Sólo dos cosas, y una de ellas con una línea:

```
struct file* efs_file(char *file_name) {
    for (int i=0; efs_files_table[i].name; i++)
        if (strcmp(efs_files_table[i].name, file_name) == 0)
            return efs_files_table + i;
    return 0;
}
```

Cómo exec lo usa

En task.c:291-313 (load_program):

1. efs_file(path) devuelve el file* de init (o falla si el path no existe / no es PROGRAM).
2. Por cada bloque de hasta 4 KiB, hace alloc_page, copia con memcpy y map_page a la VA correspondiente.

Diferencias con xv6fs

EFS no tiene:

- Inodes ni superblock.
- Bloques en disco (todo en RAM).
- Directorios (es una lista plana).
- Permisos.
- Escritura, append, truncate, delete.

Esto es a propósito: la consigna no es “filesystem completo” sino “manera de tener un binario de usuario disponible al boot”. xv6fs (que sí tiene todo lo anterior) se cubre en talleres posteriores en otras versiones del curso.

15. Programas de usuario (user/)

Qué hay

- `init.c` — el “programa init” que es el primer (y único, en 08-process) proceso de usuario.
- `edolib.c/edolib.h` — la mini-libc de EDOS. Provee `printf`, `strcpy`, `memset`, etc., más la entrada `start()`.
- `usys.s` — un stub por `syscall` que carga `a7` y hace `ecall`.
- `user.ld` — linker script con `ENTRY(start)` y `. = 0;`.
- `Makefile` — compila a rv32 estático y empaqueta.
- `mkefs.sh` — convierte los binarios en `efsfiles.c`.
- `README` — archivo de datos para probar EFS.

Cómo se compila el ejecutable

```
riscv64-unknown-elf-gcc -O -Wall -mmodel=medany -march=rv32ima -mabi=ilp32 \
  -ffreestanding -nostdlib -fno-common -fno-omit-frame-pointer \
  -fno-stack-protector -nostartfiles -static \
  -Wl,-Tuser.ld,--build-id=none -o init -e start \
  edolib.o usys.o init.c
riscv64-unknown-elf-objcopy -S -O binary init init.bin
```

Notas:

- `rv32ima` = `rv32` base + multiplicación + atómicos.
- `nostartfiles` suprime el `crt0.o` de GCC: usamos *nuestra* `start` en `edolib.c:14`.
- `-e start` explicita el entry point.
- `objcopy -O binary` elimina la cabecera ELF; el binario que termina en EFS son *instrucciones puras* listas para mapear en `VA=0`.

start y el ciclo de vida

```
extern int main();
void start(void) { exit(main()); }
```

(`edolib.c:14-17`.) La primera función definida en el `.text`, garantizada por `KEEP(*(.text.start))` en `user.ld`. Aunque acá usamos sólo la convención del nombre, en proyectos más grandes habría una sección `.text.start` explícita.

Parte III

Diferencias y similitudes con xv6

16. Tabla comparativa

Aspecto	EDOS (08-process)	xv6-riscv
ISA target	rv32ima (ilp32)	rv64 (lp64)
Paginación	Sv32, 2 niveles, página 4 KiB	Sv39, 3 niveles, página 4 KiB
Flags PTE expuestos	V, R, W, X, U	V, R, W, X, U, A, D, G
Layout kernel	1:1 desde <code>0x80000000</code> ; MMIO mapeado 1:1; sin trampoline	1:1 + trampoline en la VA más alta (en cada proceso)

Trampoline	No usa página separada; <code>s_trap/u_trap</code> viven en <code>.text</code> y se ven gracias a la copia de <code>kernel_pgtbl</code>	Página dedicada TRAMPOLINE mapeada en cada proceso y en el kernel
Trap frame	En la cima del kernel-stack del task	Página trapframe per-proc mapeada vía VA fija
PCB	<code>struct task</code> de <code>task.c</code> ; UNIFICA threads de kernel y procesos	<code>struct proc</code> ; tienen trapframe y context separados
Estados	UNUSED, CREATED, RUNNABLE, RUNNING, WAITING, ZOMBIE	UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
Scheduler	Round-robin sobre <code>tasks[]</code> , quantum=2 ticks, sin prioridades	Round-robin con sleep-locks; idéntica filosofía
Context switch	<code>context_switch</code> en <code>arch.s</code> , 14 callee-saved (rv32)	<code>swtch.S</code> , 14 callee-saved (rv64)
Locking	<code>spinlock</code> (unsigned int) con <code>push/pop_irq_off</code>	<code>spinlock</code> (con holder, name) + <code>sleeplock</code> encima
Sistema de archivos	EFS (embebido, RAM, read-only, sin directorios)	xv6fs (en disco, journaled, inodes, directorios)
Syscalls	6 (<code>exit</code> , <code>getpid</code> , <code>console_puts</code> , <code>console_putc</code> , <code>console_getc</code> , <code>sleep</code>)	~22 (incluye <code>fork</code> , <code>exec</code> , <code>wait</code> , <code>read</code> , <code>write</code> , <code>open</code> , <code>close</code> , <code>pipe</code> , <code>fstat</code> , <code>dup</code> , <code>sbrk</code> , <code>kill</code> , <code>mkdir</code> , <code>link</code> , <code>unlink</code> , <code>chdir</code>)
Convención syscall	<code>a7</code> = número, <code>a0-a6</code> = args, <code>a0</code> = ret (idéntica)	Idéntica
Driver UART	NS16550A, polling TX, RX configurable	NS16550A, IRQ-driven RX, ring buffer
SMP	Soporta hasta NCPU=4, pero <code>Makefile</code> arranca <code>-smp 1</code>	Plenamente SMP; un hart 0 inicia y los demás esperan
M-mode	EDOS hace el rol de OpenSBI: setea PMP, deleg, MPP, timer	xv6 con <code>-bios default</code> se apoya en OpenSBI; el código RISC-V en xv6 entra ya en S-mode

17. Discusión: el porqué pedagógico de cada decisión

rv32 vs. rv64. xv6-riscv eligió rv64 porque emula servidores modernos. EDOS optó por rv32 para que (a) las direcciones quepan en `uint32`, (b) las traducciones de páginas tengan sólo **dos niveles**, (c) los listados de ensamblador usen `lw/sw` (más intuitivos) y (d) la consigna “imprimir un puntero” no asuste con 8 dígitos hexa.

Sv32 vs. Sv39. Mismo motivo: dos niveles permiten dibujar la traducción completa en un slide. Sv39 tiene 3 niveles + páginas de 1 GiB y 2 MiB que quitan foco.

Sin trampoline page. xv6 mapea una página de código (`trampoline.S`) en la VA más alta tanto en kernel como en cada proceso. Esa página tiene `uservec/userret`. EDOS toma un atajo: la copia del directorio raíz del kernel en cada `pgtbl` de proceso incluye las PTEs que apuntan a las páginas del `kernel.text`, y por tanto `s_trap/u_trap` son accesibles desde el proceso (aunque sin la flag `U`, claro: sólo el hardware en modo S puede saltar ahí).

Trap frame en el kernel-stack. xv6 reserva una página `trapframe` por proceso, en una VA fija. En EDOS, el trap frame se construye *empujando* 31 registros sobre el kernel-stack del task; la macro `task_trap_frame_address(t)` es simplemente `t->kstack + PAGE_SIZE - sizeof(struct trap_frame)`. Funciona porque la página del kernel-stack ya está accesible al kernel desde cualquier proceso. La ventaja didáctica: un solo dato para entender (“la cima del stack”), sin VA mágica adicional.

EFS vs. xv6fs. xv6 tiene un FS de disco completo (inodes, journal, directorios, crash-safety). Para entender “cómo se carga el init” no hace falta nada de eso: EFS pega los binarios al kernel con `xxd`. A cambio el alumno aprende que “filesystem” no significa “ext4”; significa “mapeo nombre → bytes”.

Round-robin puro. EDOS no tiene prioridades ni MLFQ. La cátedra muestra esa complejidad en los prácticos teóricos; el kernel se queda con la forma más simple para que la atención esté en `context_switch` y la danza del lock con el scheduler.

Parte IV

Banco de preguntas de defensa

A continuación, 35 preguntas razonables agrupadas por tema, con respuestas completas que citan archivo y línea para que las puedas ir leyendo en paralelo.

18. Conceptuales generales

P1. ¿Qué es un trap y por qué nuestro kernel *necesita* uno?

Respuesta

Un *trap* es cualquier transferencia involuntaria del flujo de control desde el código actual hacia un handler en modo supervisor. En RISC-V hay tres familias: **interrupciones** asíncronas (timer, dispositivos), **excepciones** síncronas (page fault, instrucción ilegal, división por cero) y la **trap explícita** (`ecall`, que un proceso usa para pedirle al kernel un servicio).

Sin traps, el kernel **no podría existir tal como lo conocemos**: ningún proceso podría pedirle servicios al SO sin ejecutar él mismo en modo privilegiado, y el SO no podría recuperar la CPU de un proceso que no la cede voluntariamente. La existencia de traps + dos modos (U/S) es la base de la separación entre “programa de usuario” y “kernel”.

En EDOS los handlers de bajo nivel están en `trap.s`: tres de ellos (`s_trap`, `u_trap`, `m_trap`) y se ramifican en C en `trap.c::kernel_trap` y `trap.c::user_trap`.

P2. ¿Qué es un trampoline y EDOS lo usa?

Respuesta

En xv6 el *trampoline* es una página de código mapeada en una VA fija tanto en el espacio del kernel como en el de cada proceso. Contiene `uservec` y `userret` (los pasajes `U → S` y `S → U`). El nombre viene de que “ambos espacios pueden saltar en él”.

EDOS no usa una página trampoline separada. En su lugar:

- Las rutinas `s_trap` y `u_trap` viven en el segmento `.text` del kernel (`trap.s:10,89`).
- `exec` copia el directorio raíz del kernel en cada `pgtbl` de proceso (`task.c:386`). Esa copia incluye las PTEs intermedias que llevan a la `.text` del kernel.
- Por lo tanto la VA donde está `u_trap` es *traducible* (a la misma PA, vía 1:1) desde cualquier proceso. Cuando ocurre el trap y el hardware salta a `stvec = u_trap`, no hay un cambio de `satp`: el código del kernel ya está accesible.

La consecuencia es que `u_trap` **no** cambia el `satp` (es decir, sigue con el `pgtbl` del proceso) mientras hace el trabajo bajo nivel; sólo cuando entra a `user_trap` se hace `set_page_table(kernel_pgtbl)` (`trap.c:126`).

P3. ¿Qué hereda un task que viene del scheduler? ¿Cómo se entera el scheduler de dónde retomar?

Respuesta

La pieza es `context_switch` (`arch.s:90-122`). Se invoca como `context_switch(&old, &new)` con dos punteros a `struct context`. La rutina:

1. Guarda en `*old` los registros callee-saved actuales: `ra`, `sp`, `s0-s11`.
2. Restaura desde `*new` esos mismos registros.
3. Hace `ret` (jr `ra`), que salta al `ra` recién restaurado.

Por eso, cuando el scheduler hace `context_switch(&own, &task->ctx)`:

- Guardó su propio `ra` (la instrucción siguiente a la llamada) en `own.ra`. *Esa es su marca de retorno*.
- Cargó `task->ctx.ra` y saltó allí. Si la tarea nunca corrió, `ra` apunta a `start_process` (preparado en `create_task`). Si ya corrió y luego cedió, `ra` apunta justo después del `context_switch` de *sched* que la cedió.

Lo importante: el control retorna al scheduler exactamente cuando otro task hace `context_switch(&task->ctx, &cpu->ctx)`, y entonces el flujo retorna a la línea siguiente `cpu_state[cpu_id].task = NULL`; en `task.c:112-113`.

P4. ¿Qué pasa con los stacks durante el cambio de contexto?

Respuesta

`context_switch` salva y restaura el `sp` como cualquier otro callee-saved (`arch.s:93,109`). Entonces:

- Cuando se entra a `context_switch`, estamos en el stack del que cede.
- Después del `lw sp, 4(a1)` estamos en el stack del próximo. El `ret` usa ese stack.

Es seguro porque ambas estructuras `context` fueron preparadas con `sp = base del kstack + 4KiB` en algún momento previo, y la zona de stack está siempre *válidamente mapeada* en el `kernel_pgtbl` (todos los stacks viven en la RAM 1:1).

P5. ¿Cómo asegura el spinlock que no haya deadlock entre un trap y la sección crítica?

Respuesta

Porque `acquire` llama a `push_irq_off` (`spinlock.c:7-17`) antes de hacer el test-and-set. Esto deshabilita las IRQs en la CPU local. Mientras tenemos el lock, no puede llegar un trap del timer que invoque `yield` y nos deje girando para siempre. `pop_irq_off` sólo rehabilita cuando el último `release` (anidamiento contado en `noff`) lo permita.

En SMP, IRQs deshabilitadas en una CPU no garantizan exclusión entre CPUs (eso sí lo hace el spinlock); pero sí garantizan que la propia CPU no se interrumpa a sí misma estando dentro de la sección crítica.

19. Caminá el código de...

P6. Caminá el `exec` desde `create_process` hasta el primer `sret`.

Respuesta

Paso a paso, con citas:

1. `create_process("init")` (`task.c:424-441`) llama a `create_task("init", start_process)`.
2. `create_task` (`task.c:53-79`) busca un slot `UNUSED` en `tasks[]`, le asigna `kstack`, `ctx.ra=start_process`, `ctx.sp=kstack+4K`, `pgtbl=kernel_pgtbl`, estado `CREATED`.
3. `create_process` asigna `pid` y llama a `exec(t, "init", 0)`.
4. `exec` (`task.c:376-421`):
 - a) `alloc_page` para el nuevo `pgtbl`.
 - b) `memcpy(new_pgtbl, kernel_pgtbl, PAGE_SIZE)` — copia las entradas raíz del kernel.
 - c) `load_program(new_pgtbl, "init")` (`task.c:291-313`) llama `efs_file("init")` y mapea cada bloque a VAs desde 0 con `R|W|X|U`.
 - d) `alloc_page` para el user stack y `map_page` a `PROC_MAX_VA-4 KiB` con `R|W|U`.
 - e) `setup_main_args` (no hay args para `init`, pero igual configura `a0=0`, `a1=PROC_MAX_VA`).
 - f) Setea en el trap frame: `tf->pc=0`, `tf->sp=top stack user`.
 - g) Asigna `task->pgtbl = new_pgtbl` y `task->ctx.sp = (size_t) tf`.
5. `create_process` pone `state=RUNNABLE`.
6. En algún hart, el scheduler (`task.c:90-118`) encuentra el task `RUNNABLE`, lo elige y hace `context_switch` a `task->ctx`.
7. Como `ctx.ra = start_process`, el `ret` de `context_switch` cae en `start_process` (`task.c:366-372`) con `sp` apuntando al trap frame en la cima del `kstack`.
8. `start_process` libera el lock del task y llama `return_to_user_mode()`.
9. `return_to_user_mode` (`trap.c:72-101`):
 - a) `disable_interrupts()`.
 - b) `stvec = u_trap`.
 - c) `sscratch = task->kstack+4 KiB`.
 - d) `sstatus.SPP=0; SPIE=1`.
 - e) `satp = task->pgtbl` (con `sfence.vma`).
 - f) `u_trap_ret(&trap_frame)`.
10. `u_trap_ret` (`trap.s:150-192`) pone `sp=trap_frame`, restaura `sepc` desde `tf->pc=0`, restaura todos los registros (incluido `sp=tf->sp`) y hace `sret`.
11. `sret` salta a la VA 0 en **U-mode**, donde está la primera instrucción de `start` (entry de `init` según `user.ld`).

P7. Caminá la `getpid()` desde el espacio de usuario.

Respuesta

1. En `init.c`, `getpid()` es una declaración `extern int getpid(void)` de `edolib.h:24`. Su definición está en `user/usys.s:13-17`:

```
getpid:
    li a7, 1 # SYS_GETPID
    ecall
    ret
```

2. `ecall` en U-mode genera un trap con `scause=8` (`SYSCALL` en `arch.h:73`).
3. El hardware salta a `stvec = u_trap` (`trap.s:89`).
4. `u_trap` guarda registros en el `kstack` del proceso, llama `user_trap()`.
5. `user_trap` (`trap.c:107-181`) en el case `SYSCALL` (línea 146):
 - a) `enable_interrupts()`.
 - b) `syscall(task)` (en `syscall.c:73`).
 - c) `skip_trap_instruction(tf)`.
6. `syscall` lee `n = tf->a7 = 1`, indexa `syscalls_table[1]` y llama `sys_getpid(task)` (`syscall.c:16-19`), que devuelve `task->pid`.
7. `syscall_put_result(tf, result)` pone `tf->a0 = task->pid`.
8. `user_trap` retorna; `return_to_user_mode` y `u_trap_ret` restauran los registros del frame (`a0` ahora tiene el PID) y hacen `sret` a `tf->pc + 4` (`ecall` es 4 bytes).
9. En U-mode, el `ret` de `usys.s` salta de vuelta al llamador, que ve el PID en `a0` según ABI `ilp32`.

P8. Caminá un *page fault* del proceso.

Respuesta

1. El proceso accede a una VA no mapeada (o con flags incorrectos: ej. escribir una página R+X).
2. La MMU genera la excepción. `scause` toma uno de: `INSTRUCTION_PAGE_FAULT` (0xc), `LOAD_PAGE_FAULT` (0xd) o `STORE_PAGE_FAULT` (0xf) (`arch.h:75-79`). `stval` contiene la VA que falló y `sepc` la instrucción que la disparó.
3. Salta a `stvec = u_trap`.
4. `u_trap` guarda el trap frame y llama `user_trap`.
5. `user_trap` entra a uno de los `case ..._PAGE_FAULT` (`trap.c:154-162`):

```
case LOAD_PAGE_FAULT:
case LOAD_ACCESS_FAULT:
case INSTRUCTION_PAGE_FAULT:
case STORE_ACCESS_FAULT:
case STORE_PAGE_FAULT:
    printf("Task %s in CPU %d page fault. Killing task...\n", ...);
    task->exit_code = 1;
    task->killed = true;
    break;
```

6. Luego del `switch`, si `task->killed`, llama a `terminate()` (`task.c:237-259`) que pone `state=ZOMBIE`, hace `wakeup(t)` y entra a `sched`.
7. El scheduler, en su próxima iteración, ve el ZOMBIE y llama `free_resources(t)` (`task.c:82-87`) que libera el `kstack` y pone `state=UNUSED`.

Nota: EDOS *no* implementa demand paging, COW ni expansión del stack. Cualquier page fault de usuario es fatal. Es una diferencia importante con xv6 (que tampoco hace demand paging para texto, pero sí mata el proceso de forma similar; eso sí, en xv6 más recientes el stack crece con guard).

P9. Caminá un `sleep(4)`.

Respuesta

1. User-space: `sleep(4)` (`usys.s:42-45`) → `li a7, 5; ecall`.
2. Trap a `u_trap` → `user_trap` → `syscall` → `sys_sleep` (`syscall.c:48-54`): `sleep(4)` en kernel.
3. `sleep(4)` (`task.c:203-212`): `acquire(&ticks_lock)`, `task->sleep_ticks = 4`, `suspend(&ticks, &ticks_lock)`.
4. `suspend` libera `ticks_lock`, marca `state=WAITING`, `wait_condition=&ticks`, hace `sched(task)` — la tarea cede la CPU.
5. Cada timer en el hart 0 llama a `inc_ticks()` (`task.c:272-288`), que recorre las tareas `WAITING` con `wait_condition == &ticks` y decrementa su `sleep_ticks`. Cuando llega a 0, pone `state=RUNNABLE`.
6. En la siguiente iteración del scheduler, la tarea es elegible; `context_switch` vuelve a `sched` en `suspend`, que reasume `ticks_lock`, retorna, y `sys_sleep` devuelve 0.
7. `user_trap` retorna; `u_trap_ret` salta de vuelta a `ret` en `usys.s`.

P10. Caminá el camino del timer en kernel mode.

Respuesta

1. Estamos ejecutando código del kernel (por ejemplo, dentro del scheduler o de una syscall).
2. `mtime` alcanza `mtimecmp`; el hardware levanta IRQ M-mode.
3. `mtvec = m_trap`; saltamos a `m_trap` (`trap.s:198-250`): salva los `a*`, llama a `next_timer_interrupt(tp)` para reprogramar `mtimecmp`, escribe `sip = 2` (SSIP), `mret`.
4. De vuelta en S-mode, vemos pendiente la software int. `scause = 0x80000001` (TIMER_INTERRUPT).
5. Como `stvec = s_trap`, saltamos a `s_trap` (`trap.s:10`). Guarda registros en *el mismo stack* (estamos en kernel), llama `kernel_trap`.
6. `kernel_trap` (`trap.c:29-67`): en case `TIMER_INTERRUPT`, incrementa ticks (sólo hart 0), `ack_timer_interrupt()` (limpia `sip.SSIP`); si hay tarea actual y se le acabó el quantum, `yield()`.
7. Después del switch, `kernel_trap` restaura `sepc` y `sstatus` (puede haber cambiado por un `context_switch`) y retorna.
8. `s_trap` restaura registros y hace `sret` → vuelve al PC que tenía antes.

20. ¿Qué pasa si...?

P11. ¿Qué pasa si saco el primer `sfence.vma` de `set_page_table`?

Respuesta

La TLB de la CPU sigue conteniendo entradas del `satp` anterior. Cuando hagamos accesos a memoria justo después de `w_satp(...)`, la MMU resolverá esas direcciones usando las traducciones viejas, no las nuevas. En la práctica:

- Si pasamos del `pgtbl` de un proceso A al de B, accesos a las primeras instrucciones del kernel podrían traducirse contra la tabla de A (que está pendiente de flush). El resultado típico es leer instrucciones equivocadas o una excepción de violación de permisos.
- En SMP, peor: cada CPU mantiene su propia TLB; si A se movió de CPU, la TLB de la nueva CPU puede no estar sincronizada.

Es justamente lo que `sfence.vma` (sin argumentos) hace: invalida *todas* las traducciones cacheadas en la TLB local.

P12. ¿Y si saco el segundo `sfence.vma`?

Respuesta

Generalmente menos grave que sacar el primero, pero igualmente incorrecto. El segundo asegura que, después de que escribimos en una tabla de páginas (creando una nueva entrada), la próxima referencia a esa VA vea la nueva entrada en vez de un viejo cache miss/parcial. Sin el segundo, podríamos hacer `map_page` y luego acceder inmediatamente a la VA mapeada y obtener un page fault espurio, porque la TLB aún no llenó la entrada.

P13. ¿Qué pasa si no guardo `sepc` antes de llamar `user_trap`?

Respuesta

`sepc` es un CSR *compartido* para el modo S. Si durante `user_trap` ocurre otro trap (por ejemplo, una syscall del propio kernel — no, eso no pasa, pero un page fault del kernel sí), `sepc` se sobrescribe con el nuevo PC. Al volver, `sret` saltaría al lugar equivocado. El kernel necesita “apropiarse” del `sepc` actual y guardarlo. En `u_trap` (`trap.s:136-137`) eso se hace antes de la llamada:

```
csrr a0, sepc
sw a0, 30 * 4 (sp) # tf->pc
```

Y al retornar, `u_trap_ret` (`trap.s:156-157`) restaura:

```
lw a0, 30 * 4 (sp)
csrw sepc, a0
```

Si lo olvidamos, después de un timer en U-mode el `sret` final saltaría a la dirección del último trap que haya ocurrido.

P14. ¿Y si el spinlock no deshabilita interrupciones?

Respuesta

Deadlock clásico de tipo *lock-to-self*:

1. Llamamos `acquire(&L)`. Lo obtenemos.
2. Llega un timer. `s_trap` → `kernel_trap` → `yield`. `yield` a su vez quiere usar `tasks[i].lock` para cambiar de estado.
3. Si el handler del trap necesita el lock L (o cualquier lock que esté actualmente tomado), giraría esperándolo *en la misma CPU que ya lo tiene*. **Deadlock irreparable.**

Aún cuando los locks no se solapan, hay un escenario más sutil: si un `wakeup` se dispara desde un handler de trap mientras nosotros tenemos el lock de `tasks[i]`, y el `wakeup` intenta tomar ese mismo lock, esperamos eternamente.

Por eso `acquire` llama a `push_irq_off` antes del test-and-set (`spinlock.c:29-30`).

P15. ¿Qué pasa si `exec` no copia el kernel `pgtbl`?

Respuesta

El proceso quedaría con un `pgtbl` que mapea su código y stack, pero *nada* del kernel. Apenas saliera por `sret` a `u_trap_ret` (que está en `.text` del kernel), generaría un `INSTRUCTION_PAGE_FAULT` porque el PC apunta a una VA no mapeada en su tabla.

Lo más temprano que se vería es al ocurrir el primer trap: la CPU salta a `stvec=u_trap`. Esa dirección sería inválida en el `pgtbl` actual del proceso. Resultado: una *trap loop* infinita o una excepción doble que termina en hardware reset.

La copia `memcpy(new_pgtbl, kernel_pgtbl, PAGE_SIZE)` en `task.c:386` es lo que evita esto.

P16. ¿Qué pasaría si NCPU no coincide con `-smp N` de QEMU?

Respuesta

NCPU dimensiona `cpus_state[NCPU]` (`task.c:14`). Si QEMU lanza más harts que NCPU, `cpuid()` devolvería un índice fuera de rango y todas las operaciones sobre `cpus_state[cpu]` corromperían memoria.

Si QEMU lanza menos, no pasa nada: los slots extra de `cpus_state[]` quedan sin usar.

En el Makefile actual se lanza `-smp 1`, así que sólo el hart 0 corre y el resto del array está ocioso.

21. Cálculos sobre paginación Sv32

P17. ¿Cuántas entradas hay por tabla en Sv32? ¿Cuánto ocupa cada tabla?

Respuesta

Cada tabla (directorio raíz o tabla hoja) tiene $2^{10} = 1024$ **entradas**. Cada entrada (PTE) es de **4 bytes**. Por lo tanto una tabla ocupa exactamente **4096 B = 1 página**. Es por eso que `alloc_page` alcanza para crear un directorio o una tabla hoja: una llamada = una tabla.

P18. Traducir manualmente la VA `0x80004ABC` con `kernel_pgtbl`.

Respuesta

- `VPN[1]` = bits 31..22 de `0x80004ABC` = `0b10_0000_0000` = `0x200` = 512.
- `VPN[0]` = bits 21..12 = `0b00_0000_0100` = `0x4` = 4.
- `Offset` = bits 11..0 = `0xABC` = 2748.

Buscamos `kernel_pgtbl[512]`. `map_kernel_memory` mapeó `ktext_start = 0x80000000` con tamaño = `__text_end - __kernel_start`, R+X, 1:1. Como `0x80004ABC` $\in$ $[0x80000000, \text{__text_end})$, si `__text_end > 0x80005000`, la VA está mapeada con PTE para frame físico `0x80004` (es decir `PPN=0x80004`, escrito `0x80004` $\ll$ 10 | (R|X|V) = `0x20001000` | `0xb...`).

Bajando dos niveles:

- `root_table[512]` apunta a la tabla hoja con su PPN y V=1.
- En esa tabla hoja, la entrada [4] tiene `PPN = 0x80004`, flags R+X+V.
- `PA = (PPN  $\ll$  12) + offset = 0x80004000 + 0xABC = 0x80004ABC.`

Como esperábamos para una región 1:1.

P19. ¿Cuánta memoria consume el *conjunto* de tablas de páginas de *init* en EDOS?

Respuesta

Estimemos suponiendo que el *init.bin* pesa $\sim 1\,300$ B (redondeamos a 1 página de código/datos) y la *stack* ocupa 1 página más en `PROC_MAX_VA-4 KiB`.

- Directorio raíz: 1 página (*exec* la asigna en `task.c:381`).
- Tabla hoja para `VPN[1]=0` (cubre la VA 0): 1 página (la crea *get_pte* cuando se mapea `VA=0`).
- Tabla hoja para `VPN[1]=511` (cubre la VA `0x7FC00000..0x80000000`, donde está el *stack* en `0x7FFFF000`): 1 página.

Total para el proceso: 3 páginas = 12 KiB (más las PTs heredadas del kernel — *compartidas* con todos los procesos, no se cuentan).

Si el *init* fuera más grande y cruzara una frontera de 4 MiB (`VPN[1]` distinto), necesitaríamos más tablas hoja.

P20. ¿Cuál es el máximo espacio de direcciones de un proceso EDOS?

Respuesta

`PROC_MAX_VA = 0x80000000 = 2 GiB` (`task.h:20`). Eso es de 0 a `0x7FFFFFFF` (excluido `0x80000000`).

¿Por qué no los 4 GiB completos del Sv32? Porque desde `0x80000000` hacia arriba está mapeada la RAM del kernel 1:1 en `kernel_pgtbl`, y *exec* copia esa porción *en cada proceso*. Si dejáramos al usuario usar esas VAs, chocaría con código del kernel y la separación user/kernel perdería sentido.

Es exactamente la decisión que xv6 toma con `MAXVA` y `KERNBASE`, aunque allí los números son distintos (Sv39).

22. Trade-offs

P21. ¿Por qué Sv32 y no Sv39?

Respuesta

La cátedra ya tiene que enseñar paginación, interrupciones, traps, syscalls, planificación y filesystem en un cuatrimestre. Sv39 agrega:

- Tercer nivel (`PD` $\rightarrow$ `PT` inter. $\rightarrow$ `PT` hoja).
- Páginas de **4 KiB, 2 MiB y 1 GiB** (megapages y gigapages) que complican el formato de PTE.
- Direcciones virtuales con *sign extension* a 64 b (la regla de bits `63..39 = bit 38`).

Ninguno de esos detalles es central para entender “cómo funciona la traducción de VA”. Sv32 da exactamente lo mismo conceptualmente con la mitad del esfuerzo cognitivo.

P22. ¿Por qué round-robin y no MLFQ o prioridades?

Respuesta

Por la misma razón que Sv32: simplicidad. Un MLFQ requiere:

- Estructuras de colas múltiples.
- Reglas de subida/bajada de prioridad.
- Tracking del tiempo de CPU por tarea (más allá del cuanto).

Para que el alumno entienda “cambio de contexto”, “preemption”, “estado del PCB” y “yield”, round-robin alcanza. Las complicaciones de planificación se ven en teoría y en los prácticos de scheduling (Práctico 4), no en EDOS.

P23. ¿Por qué EFS y no xv6fs?

Respuesta

Tres razones:

1. **Boot rápido:** no hay que montar un disco virtio, leer superblock, parsear inodes... basta `xxd -i` y un `efs_files_table[]`.
2. **Foco:** el contenido conceptual de “filesystem” en EDOS es “mapeo nombre → bytes”. EFS lo cubre. Los detalles de journaling, FAT, inodes, etc., se cubren en prácticos teóricos.
3. **Reproducibilidad:** como el FS vive en el binario, no hay que distribuir una imagen de disco aparte.

P24. ¿Por qué no hay fork en EDOS?

Respuesta

Porque **fork** requiere **copiar el espacio de direcciones del padre** (o implementar COW). Eso significa caminar la page table del padre y, para cada PTE válida con `U=1`, o bien:

- Asignar una página nueva, hacer `memcpy`, mapearla en el pgtbl del hijo, o
- Marcar la PTE como solo-lectura en ambos pgtbls y manejar el page fault de COW.

Ambas opciones son “ejercicios para el lector” que pueden agregarse en talleres posteriores. La versión 08-process se contenta con `create_process(path)`, que es esencialmente `fork + exec` fusionados.

23. Comparativas con xv6

P25. ¿Cómo se diferencia el manejo de la TLB?

Respuesta

Ambos hacen `sfence.vma` antes y después de cambiar `satp`. La diferencia es que xv6 tiene una rutina explícita `kvminithart` llamada *en cada hart después de boot* para cargar `satp` apuntando al `kernel_pgtbl`; en EDOS el equivalente es `enable_paging()` en `kmain.c:28`, también llamado por cada hart antes de entrar al `scheduler`.

xv6 mantiene además la VA **TRAMPOLINE** mapeada en todos los pgtbls, lo que le permite hacer `satp` switches *en ensamblador* sin perder el código que ejecuta. EDOS evita esto copiando el directorio del kernel en cada pgtbl de proceso.

P26. ¿Qué hace xv6 con el trapframe que EDOS hace distinto?

Respuesta

xv6:

- Reserva una página *específica* para el trapframe de cada proceso, mapeada en una VA fija (TRAPFRAME = TRAMPOLINE - PGSIZE).
- `trampoline.S::uservec` accede a esa VA para guardar registros, sin tocar el stack del proceso.
- Después de eso, cambia `satp` al kernel `pgtbl` y salta al handler en C.

EDOS:

- Construye el trapframe *en la cima del kernel-stack* del task. Su dirección es `kstack + 4K - sizeof(tf)`.
 - No cambia `satp` en `u_trap`; sigue con el `pgtbl` del proceso (que tiene los mapeos del kernel copiados).
 - Cambia `satp` dentro de `user_trap` (`trap.c:126`) sólo cuando ya está manejando la causa.
- La razón didáctica: EDOS prefiere “todo está en la pila” a “buscá la página mágica”.

P27. ¿Cómo se diferencia el sleep-lock de xv6 con suspend en EDOS?

Respuesta

xv6 tiene **dos** tipos de locks:

- `struct spinlock`: el clásico, no duerme.
- `struct sleeplock`: encima de `spinlock`, con `acquiresleep/releasesleep`. Si no se puede tomar, llama a `sleep(&slk->locked, &slk->lk)`, que pone al proceso SLEEPING hasta que el dueño lo libere y llame `wakeup`.

EDOS tiene *sólo* `spinlock` más `suspend/wakeup` (`task.c:163,186`). Lo segundo es construible *encima de* lo primero — y de hecho la API `suspend(condition, lk)` es funcionalmente equivalente a un sleep-lock que duerme en `condition` y reasume el lock al volver. EDOS no encapsula el patrón en un tipo separado.

P28. ¿Qué syscalls tiene xv6 que no tiene EDOS?

Respuesta

xv6 expone alrededor de 22 syscalls; EDOS, 6. Las que faltan en EDOS:

- `fork`, `wait`: no hay relación padre-hijo real en EDOS.
- `exec`: existe en kernel (`task.c:376`) pero no como syscall.
- `read`, `write`, `open`, `close`, `dup`, `fstat`: no hay descriptores de archivo en EDOS.
- `pipe`: no hay pipes.
- `kill`: no hay señales.
- `mkdir`, `link`, `unlink`, `chdir`: el FS de EDOS no soporta jerarquía ni escrituras.
- `sbrk`: no hay heap manageable en EDOS (el stack es de tamaño fijo, una página).

A cambio EDOS tiene unas pocas convenientes (`console_puts/ console_putc` explícitas), que xv6 implementa vía `write(1, ...)`.

24. Detalles finos del código

P29. ¿Por qué `m_trap` pone los `a*` en 0 después de salvarlos?

Respuesta

Mirá `trap.s:224-239`:

```
lw a0, 0 * 4 (sp)
sw x0, 0 * 4 (sp)
\dots
```

Restaura y *escribe cero* sobre el slot del stack que acaba de leer. ¿Para qué? Para que, si después el código de un proceso de usuario lee el stack del kernel (por algún error del SO o por una vulnerabilidad de filtración), no encuentre valores identificables. Es una mitigación defensiva preventiva, mínima pero presente.

P30. ¿Por qué `u_trap` hace `csrrw sp, sscratch, sp`?

Respuesta

`csrrw rd, csr, rs1` es “intercambio atómico”: escribe `rs1` en `csr` y deja en `rd` el valor previo. La instrucción aquí cambia `sp` (que tenía el SP de usuario) por `sscratch` (que tenía la base del kstack del proceso), en **una sola instrucción**. Esto es importante porque, al entrar al trap, todavía estamos con stack de usuario; cualquier escritura previa al swap rompería el stack del proceso.

Después del swap, `sp` apunta al kstack y `sscratch` al user-stack. La siguiente instrucción reserva espacio en kstack para el trap frame, y unas líneas más abajo (`trap.s:132-133`) `csrr a0, sscratch; sw a0, 29*4(sp)` guarda el SP de usuario en el frame.

P31. ¿Por qué `create_task` hace `memset(&ctx, 0, sizeof ctx)` antes de asignar `ra`?

Respuesta

Para que los registros `s0-s11` y `sp` en el contexto inicial estén en 0. Cuando `context_switch` los cargue al programar la primera ejecución, esos `s*` quedarán en cero (que es lo que correspondería para una función recién entrada). Sin el `memset`, leeríamos basura del slot `tasks[i]` que tal vez quedó de una tarea previa `UNUSED`.

P32. ¿Por qué `exec` libera la página vieja *después* de mapear la nueva?

Respuesta

Por seguridad: si la asignación o el `load_program` fallan (`goto error`), todavía conservamos la tabla del proceso original y podemos devolver `false` sin haberlo dejado en un estado inválido.

La lógica está en `task.c:406-411`:

```
if (is_process(task)) {
    unmap_region(task->pgtbl, 0, PROC_MAX_VA, true);
    free_page(task->pgtbl);
}
task->pgtbl = new_pgtbl;
```

Sólo después de que `new_pgtbl` está completo y validado liberamos el viejo. Es el patrón *prepare-then-swap* típico.

P33. ¿Cuándo se incrementa `ticks` y por qué sólo el hart 0?

Respuesta

En `trap.c:46-48` (en `kernel_trap`) y `trap.c:133-135` (en `user_trap`), dentro del caso `TIMER_INTERRUPT`, se hace:

```
if (cpu_id == 0)
    inc_ticks();
```

Cada hart tiene su propio `mtimecmp`, así que recibirían su propio timer y cada uno incrementaría `ticks`, llevando la cuenta al *múltiplo del número de harts*. Para mantener “`ticks` = tiempo real”, sólo el hart 0 incrementa.

Es una decisión simple: en `xv6` hay una similar (sólo el hart 0 mantiene la noción de tiempo).

P34. ¿Qué hace `set_u_previous_mode` y por qué hace falta?

Respuesta

`arch.h:411-417`:

```
static inline void set_u_previous_mode(void) {
    size_t status = cpu_status();
    status &= ~(1 << 8); // clear sstatus.SPP
    status |= (1 << 5); // sstatus.SPIE = 1
    set_cpu_status(status);
}
```

`sstatus.SPP` (Supervisor Previous Privilege) en 0 indica “el modo previo era U”, así que el próximo `sret` bajará a U-mode. `sstatus.SPIE` (Previous Interrupt Enable) en 1 indica “cuando se ejecute `sret`, hay que dejar `SIE=1`” (es decir, interrupciones habilitadas en U-mode).

Sin esta función, después del primer `return_to_user_mode` `sret` no haría el cambio de modo correcto.

P35. ¿Por qué sleep suspende sobre &ticks y no sobre el propio task?

Respuesta

Porque la condición no es “este task termine”, sino “*los ticks alcancen un valor*”. Toda tarea durmiendo por tiempo comparte la misma fuente de wakeups: el timer en `inc_ticks`. `inc_ticks` (`task.c:272-288`) hace un solo recorrido de la tabla, y para todas las tareas con `wait_condition == &ticks` decrementa su `sleep_ticks`.

Si se durmiera sobre un canal por tarea, `inc_ticks` tendría que enviar wakeups individuales, lo cual es más caro y no agrega claridad.

Parte V

Cheat sheet final

25. Registros RISC-V relevantes (1 cara)

Reg./CSR	Modo	Rol
<code>x0</code>	—	Siempre cero (no se puede escribir).
<code>ra</code>	—	Return address. Es el primer registro guardado por <code>context_switch</code> .
<code>sp</code>	—	Stack pointer. Cada hart arranca con un stack propio en <code>__stack0</code> .
<code>gp</code>	—	Global pointer (set por el linker).
<code>tp</code>	—	En EDOS, el <i>hartid</i> . Se setea en <code>arch.s:14</code> .
<code>a0-a7</code>	—	Argumentos y resultado. <code>a0</code> también porta el retorno de syscalls.
<code>s0-s11</code>	—	Callee-saved. Es lo que <code>context_switch</code> salva además de <code>ra</code> y <code>sp</code> .
<code>t0-t6</code>	—	Caller-saved. Sólo se salvan en el trap frame.
<code>mhartid</code>	M	Identifica el hart. Leído en boot.
<code>mstatus</code>	M	MPP (modo previo), MIE (habilita IRQs M).
<code>mepc</code>	M	PC para <code>mret</code> .
<code>mtvec</code>	M	Handler M-mode (en EDOS, <code>m_trap</code>).
<code>mie</code>	M	Bit MTIE habilita IRQ del timer.
<code>medeleg / mideleg</code>	M	Delega exc./int. a S.
<code>sstatus</code>	S	SIE (habilita IRQs), SPP (modo previo: 0=U, 1=S), SPIE (estado previo de SIE).
<code>sepc</code>	S	PC en el trap, restaurado por <code>sret</code> .
<code>scause</code>	S	Causa del trap (bit 31 = interrupción).
<code>stval</code>	S	Dirección que causó el fault (page faults).
<code>stvec</code>	S	Handler S-mode (en EDOS, <code>s_trap</code> o <code>u_trap</code>).
<code>sscratch</code>	S	“Bolsillo” del kernel. En EDOS: base + 4 KiB del kstack del proceso.
<code>satp</code>	S	Page table register. <code>0x80000000 PPN(root)</code> en Sv32.
<code>sie / sip</code>	S	Habilitación / pendiente de IRQs S-mode.
<code>pmpcfg0, pmpaddr0</code>	M	Configuración del Physical Memory Protection.
<code>mtime / mtimecmp[h]</code>	M	Timer (MMIO en CLINT).

26. Formato de PTE Sv32

bits 31..10										0
PPN (22 bits)										
	res	D	A	G	U	X	W	R	V	

EDOS sólo usa **V, R, W, X, U** (`arch.h:175-179`). En Sv32 “oficial” hay además **A** (accessed), **D** (dirty) y **G** (global) que el hardware actualiza, pero QEMU no los gestiona y EDOS no los consulta.

Reglas: V debe estar siempre en 1 para una entrada viva. Si R|W|X = 0, la entrada es un puntero a la próxima tabla (no es hoja). Si alguno de R/W/X está prendido, es hoja.

27. Secuencia mínima de boot

1. boot en arch.s: PMP, MPP=S, deleg, sie, mret a supervisor.
2. supervisor: en arch.s: stvec=s_trap, kernel_main.
3. kernel_main en kmain.c: hart 0 hace init_kalloc, init_vm, create_process("init"), publica ready=1.
4. Todos los harts: enable_paging, scheduler.

28. Secuencia del primer scheduling de init

1. Scheduler ve tasks[k].state == RUNNABLE con ctx.ra=start_process.
2. context_switch → ret a start_process.
3. start_process: release(&t->lock); return_to_user_mode();
4. return_to_user_mode: stvec=u_trap, sscratch=kstack+4K, SPP=0, SPIE=1, satp=task->pgtbl, u_trap_ret(tf);
5. u_trap_ret: sp=tf, sepc=tf->pc=0, restaurar regs, sret.
6. sret → U-mode @ pc=0 (entry start de init).

29. Secuencia de una syscall

1. User: li a7, N; ecall.
2. HW → u_trap: swap sp/sscratch, salvar regs, llamar user_trap.
3. user_trap: satp=kernel_pgtbl, stvec=s_trap, dispatch por scause=8, syscall(task), skip_trap_instruction.
4. syscall: lee a7, llama syscalls_table[N](task), escribe tf->a0.
5. return_to_user_mode → u_trap_ret → sret.

30. Comandos útiles para QEMU + GDB

```
# Compilar y correr
make
make qemu # arranca QEMU con -smp 1

# Salir de QEMU
Ctrl-A, x

# Ver el ensamblado del kernel
less kernel.asm

# Ver el ensamblado de init
less user/init.asm

# Debug con GDB:
# Terminal 1:
qemu-system-riscv32 -machine virt -bios none -nographic -smp 1 \
    -kernel kernel -s -S
# (-s = gdbserver en :1234, -S = freezar al inicio)

# Terminal 2:
riscv64-unknown-elf-gdb kernel
(gdb) target remote :1234
```

```

(gdb) set arch riscv:rv32
(gdb) b kernel_main
(gdb) c

# Para ver registros CSR:
(gdb) info registers
(gdb) p/x $satp
(gdb) p/x $sepc

# Para imprimir memoria como un trap_frame:
(gdb) p *(struct trap_frame *)(task->kstack + 4096 - sizeof(struct trap_frame))

# Para ver dónde está mapeada una VA en el pgtbl del proceso:
(gdb) p/x ((unsigned int*)task->pgtbl)[0] # PDE[0]
(gdb) p/x ((unsigned int*)pte_pa(PDE[0]))[0] # PTE[0] de la hoja

```

31. Tabla mental rápida

Pregunta urgente	Mirar en...	Línea
“Cómo se monta un PTE”	arch.c	69-73
“Cómo entra al kernel”	arch.s	11-83
“Qué hace el timer”	trap.s + trap.c	198 + 45
“Cómo cede la CPU una tarea”	task.c	140-158
“Cómo cambia entre tareas”	arch.s	90-122
“Cómo se carga un programa”	task.c	376-421
“Por qué hay un sscratch	trap.s + trap.c	93 + 84
“Cómo despierta sleep”	task.c	272-288
“Dónde está la tabla de syscalls”	syscall.c	63-70
“Cómo se inicializa la VM”	vm.c + arch.c	6 + 97